Exercises: Chapter 03

Hello there! I'm Dr. Chuck, your Python tutor. It’s great to have you here.

You're not just learning Python to see syntax rules on a screen; you want to build systems and become an **AI Engineer**. That is the absolute best mindset to have. In my book *Python for Everybody*, I always say that computers want to help us, but they speak a very simple, literal language. Our job is to learn how to write the "recipes" (programs) that guide them to handle data for us.

Since you've studied **Variables, Expressions, Statements, Conditional Execution, and Functions**, we have enough blocks to build real infrastructure. In AI and systems engineering, we deal heavily with token costs, response latency, prompt truncation, and routing logic.

Let's do some drills. We will start with **Basic/Intermediate drills** to lock down your syntax reflexes, and then move straight into **Advanced Exercises** framed as real micro-problems an AI Engineer would face.

Phase 1: Basic to Intermediate Drills

Drill 1: The Token Budget Meter (Variables & Expressions)

The Setup: When sending text to a Large Language Model (LLM), we are billed per token. As a rule of thumb, 1 token is roughly 4 characters of English text.

- **Your Task:** Write a short script that takes a long string variable representing a prompt (user_prompt). Calculate the approximate token count based on its character length. Then, calculate the total cost if 1,000 tokens cost exactly $0.002. Print both values.

Drill 2: API Gateway Router (Conditional Execution)

The Setup: In a production pipeline, you don't always want to send requests to the most expensive model if a cheaper one can handle it.

- **Your Task:** Create a variable prompt_length (representing the number of characters in a query). Write a conditional statement (if-elif-else) that handles routing:
    - If the length is greater than 5,000 characters, print: "Route to: GPT-4 (Long Context Edition)"
    - If the length is between 1,000 and 5,000 characters (inclusive), print: "Route to: Claude Instant"

- If the length is less than 1,000 characters but greater than 0, print: "Route to: Local Llama-3 Micro"
- If the length is 0, print: "Error: Empty Prompt Received"

Drill 3: Temperature Guard (Functions & Logic)

The Setup: In LLM APIs, the temperature parameter controls creativity (ranging from 0.0 for deterministic to 2.0 for highly random). If an engineering teammate accidentally passes a negative number or a value way too high, your system could crash.

- **Your Task:** Write a function called sanitize_temperature(temp).
  - It should accept a single argument temp (a float).
  - If temp is less than 0.0, it must force it to 0.0 and print a warning.
  - If temp is greater than 2.0, it must force it to 2.0 and print a warning.
  - Otherwise, it leaves it unchanged.
  - The function should return the cleaned value. Test it with inputs like -0.5, 1.2, and 2.5.

Phase 2: Advanced AI Systems Exercises

Now let's step it up. These exercises blend functions, nested conditional evaluations, type conversions, and expressions into practical micro-architectures.

Exercise 4: Dynamic Agent Cost Evaluator & Guardrail System

The System Goal: You are building an autonomous AI Agent system. The agent loops through tasks, but if a task is too expensive or exceeds the user's maximum spending ceiling, your system must halt execution immediately to avoid an infinite loop that drains the user's bank account.

Your Task: Write a function named evaluate_agent_run that takes three parameters:

1. input_tokens (integer)
2. output_tokens (integer)
3. max_budget_usd (float)

Rules of the System:

- Input tokens cost $0.0015 per **1,000 tokens**.

- Output tokens cost $0.0020 per **1,000 tokens**.
- Calculate the total execution cost using an expression.
- If the total execution cost exceeds max_budget_usd, return the string: "REJECTED: Budget exceeded. Cost would be $X.XXXX" (where X.XXXX is the calculated cost).
- If the cost is within budget, return the string: "APPROVED: Executing task. Estimated cost: $X.XXXX"

Dr. Chuck's Hint: Watch out for integer division! Make sure your division expressions retain float precision so you don't accidentally zero out your costs.

Exercise 5: Multi-Modal Context Window Sharder (The Truncation Algorithm)

The System Goal: When dealing with Retrieval-Augmented Generation (RAG) or feeding large database schemas into an LLM context window, you often hit a hard wall known as the Context Limit. If your data is too big, the API throws a 400 Bad Request error. You need to write the safety logic that truncates code context or splits it cleanly.

Your Task: Write a single, highly structured function called optimize_context_window that takes three inputs:

1. system_instruction (string)
2. retrieved_chunk (string)
3. max_allowed_chars (integer)

Operational Logic to Implement:

1. Calculate the absolute minimum overhead: the character length of the system_instruction.
2. If the system_instruction alone is already longer than max_allowed_chars, the system is fatally misconfigured. Return: "FATAL: System instruction exceeds entire context window limit."
3. Calculate how much remaining room is left for the retrieved_chunk (i.e., max_allowed_chars - len(system_instruction)).
4. If the length of retrieved_chunk fits perfectly within that remaining room, no changes are needed! Return a concatenated string: system_instruction + "\n" + retrieved_chunk.

5. If the retrieved_chunk is too large for the remaining room, you must safely truncate it right at the boundary limit. Cut the string down to fit the exact remaining space, append "[TRUNCATED]" to the end of it, and return the combined text.

How to approach this:

Take your time. Fire up your terminal, open VS Code or a Jupyter Notebook, and start drafting your solutions.

Don't worry about making mistakes on your first try—getting trackback errors is how our brain patterns map out Python's strict logic blocks. Paste your code drafts for these exercises right here when you're ready, and we will look through them together like code reviewers on an engineering team! Which one would you like to sketch out first?